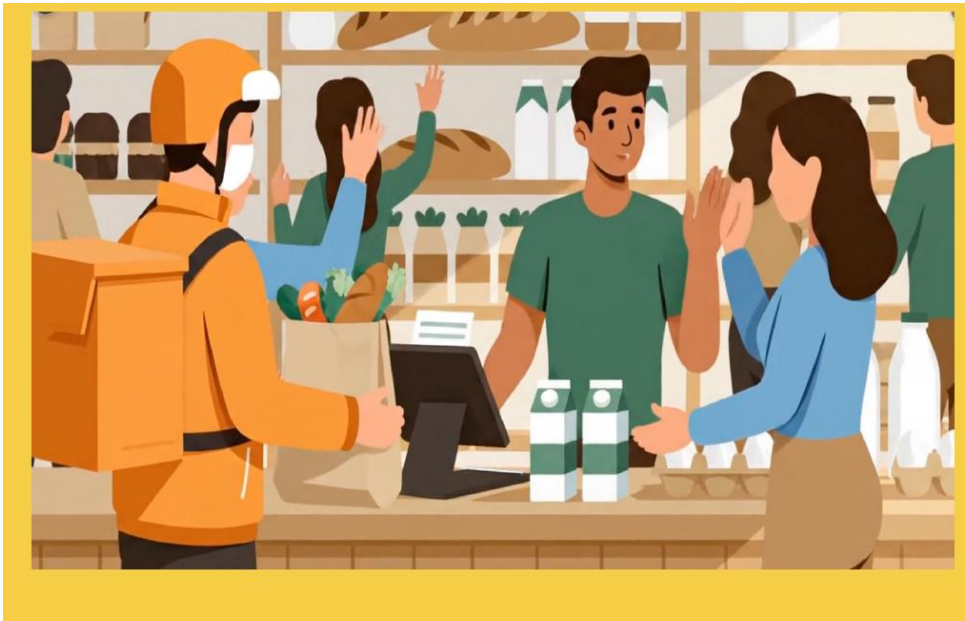


SONIK

Smart Retail Management System

A unified, AI-assisted platform that replaces the notebooks small grocery stores use for billing, inventory, credit, and supplier management.



Role	Product Owner — end-to-end PM lifecycle: user research, PRDs, MoSCoW prioritisation, sprint execution, AI assistant feature
Team	Team 062 — “SoftEx” (4 members)
Duration	February 2026 – April 2026 · 5 milestones across 2 build sprints
Tools	FastAPI, Vue 3, SQLite/MySQL, scikit-learn, Groq LLM, Razorpay, Figma, Jira/Kanban, GitHub

Aanchal Routela (23F2004126) · Navirah Mohsin (23F2002090) · Md. Faiz Khan (23F2000288) · Abhilash Kumar Das (21F3003066)
IITM Online BS Degree Program · Indian Institute of Technology, Madras, Chennai · Tamil Nadu, India, 600036

Contents

- 01. Problem Statement
- 02. Target Users
- 03. User Research
- 04. Product Goal
- 05. User Journey / Flow
- 06. Wireframes & UX Decisions
- 07. Feature Prioritisation
- 08. PRD Highlights
- 09. AI Feature
- 10. AI Architecture
- 11. Evaluation & Metrics
- 12. Challenges & Trade-offs
- 13. Results & Learnings
- 14. Future Improvements (V1 → V2)
- 15. My Contribution

01. Problem Statement

Small retail stores run on notebooks, not systems — and that gap is where Sonik starts.

Sonik was scoped around a problem experienced daily by small grocery and retail owners across India — represented in this project by **Mr. Vinod Singh**, a grocery-store owner with 12–13 years of experience running a neighbourhood shop. Vinod runs every part of the business manually: billing, stock counts, supplier coordination, customer credit (“udhaari”), and expenses are all tracked in physical notebooks and informal, in-head calculations.

Who experiences it

Small and medium grocery/retail owners who cannot justify the cost or complexity of enterprise retail-management software, and who currently depend on notebooks and memory to run day-to-day operations.

Why it matters

Manual operations create compounding, invisible costs: inaccurate stock estimates, delayed credit recovery, no product-wise profit visibility, and weak cash-flow control. None of these show up as a single dramatic failure — they quietly erode margin every day.

Why existing tools fall short

Most retail tools available to small stores focus on basic record-keeping. They don't offer predictive insight, natural-language access to business data, or affordable AI-assisted decision-making — the things that would actually change how a small retailer makes decisions rather than just digitising the same notebook.



Fig 1.1 — The shop profile, user stories, and challenges that framed the problem (project kickoff deck).

02. Target Users

User identification followed a primary / secondary / tertiary structure, matching how work is actually distributed in a single-owner shop today — and how it is expected to scale as Vinod hires help.

Tier	User	Needs	Pain points today
Primary	Store Owner (Vinod) — sole operator	One place to see stock, credit, and profit without notebooks	No profit-margin visibility; manual credit tracking; 1–2% bad debt; over/understocking
Secondary	Employees / future helpers	A simple interface for billing and stock tasks, with shift accountability	No shift or attendance records; billing errors can't be traced to a person
Tertiary	Customers	Accurate bills; fair, trackable credit terms	Billing inaccuracies; disputes over credit balances
Tertiary	Suppliers	Timely, traceable payments	Manual, delayed payment tracking

A simple persona anchored the whole system:

Persona — Vinod Singh, Store Owner

12–13 years running a grocery shop · manages billing, inventory, supplier coordination, credit, and expenses alone · open to hiring helpers as the business scales · wants visibility, not more paperwork.

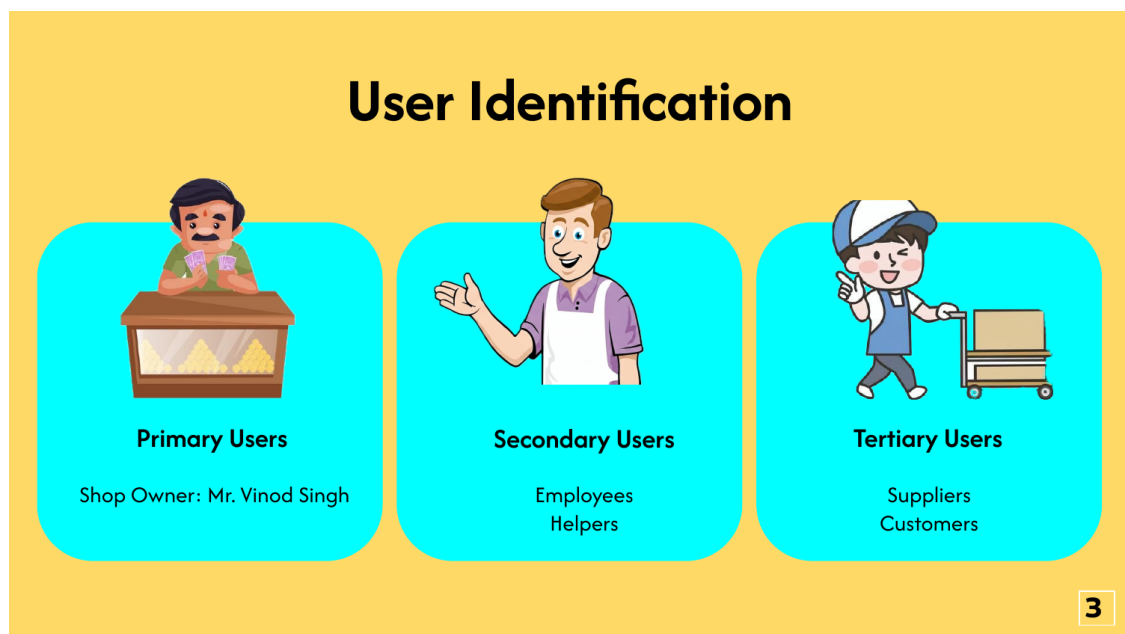


Fig 2.1 — Primary, secondary, and tertiary users identified for Sonik.

03. User Research

Method

A semi-structured interview plus direct observation of the store owner, Mr. Vinod Singh (12–13 years' experience, grocery retail). The interview deliberately focused on identifying operational problems without suggesting technology solutions, so findings would reflect real workflow pain rather than pre-baked feature ideas.

Key findings → product implications

Finding	Evidence	Product implication
No structured profit calculation	Manual notebook math; no product-wise margin visibility	Finance module with automatic daily/monthly profit calculation
Credit (udhaari) is hard to manage	Physical notebooks; 1–2% bad debt; delayed payments	Credit module with per-customer balances, limits, and risk levels
Inventory is estimated, not tracked	No expiry tracking; no reorder mechanism; over/understocking	Inventory module with expiry dates, reorder levels, low-stock alerts
Milk subscriptions are logged by hand	Risk of quantity miscalculation; billing inaccuracies	Milk Subscriber + Daily Entry module with monthly invoicing
No structured expense logging	Cash doesn't reconcile with expected sales	Expense tracking tied into the Finance dashboard
No employee accountability (emerging concern)	No shift or attendance record; no link between errors and staff	Shift login + sales-by-employee reporting

04. Product Goal

User goal

Give the store owner one place to bill, track stock, manage credit, and understand the business — without a notebook.

Business / product goal

Replace fragmented manual processes with a single integrated, data-driven, AI-assisted retail platform that a small store can realistically run on.

Objectives

- Centralised retail management — billing, inventory, credit, suppliers, and employees in one system.
- Efficient inventory & billing, replacing manual stock counts and paper bills.
- Customer credit (udhaari) tracking with visibility into overdue balances.
- Data-driven insights — sales trends, demand, and credit risk surfaced automatically.
- An AI assistant that can answer business questions in plain English against live data.
- A scalable, REST-API-based system that can move from SQLite to MySQL without a rewrite.

05. User Journey / Flow

The flow below covers the core loop every role in Sonik repeats — from opening the app to acting on an AI-surfaced insight:

```
User (Owner / Employee)
  |
  v
Opens Sonik ---> Logs in as Owner or Employee (X-User-ID session)
  |
  v
Selects a module: Billing / Inventory / Credit / Milk / Suppliers / Finance
  |
  v
Provides data: cart items, stock update, payment, expense entry
  |
  v
FastAPI backend validates the request, writes to SQLite/MySQL,
and recomputes dashboard KPIs
  |
  v
AI Assistant / Dashboard Insights: a plain-English question
is turned into SQL and run against the live database
  |
  v
Result: a receipt, report, or insight card is shown to the user
  |
  v
User action: restock, follow up overdue credit, pay a supplier, etc.
```

06. Wireframes & UX Decisions

Screens below are from the working Vue 3 prototype. Several were deliberately changed after real users — not just the team — tried them, which is called out for each one.

Login — role-aware from day one

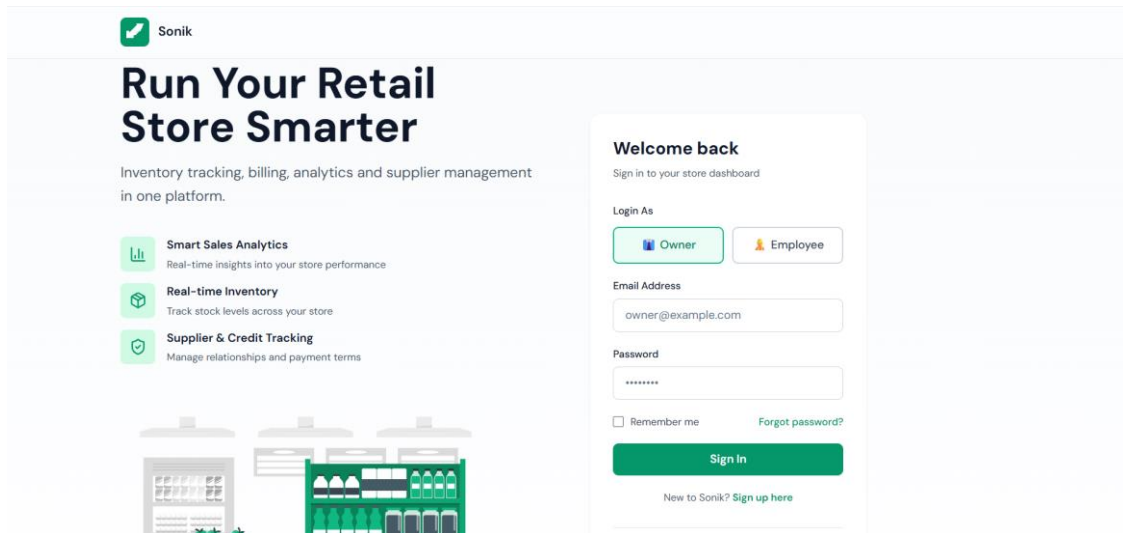


Fig 6.1 — Login screen with an explicit Owner / Employee switch.

Owner and Employee were split at login rather than handled with a single role flag, because the persona work in Section 02 made clear the two roles need almost entirely different screens (full analytics vs. billing + shift only).

Owner Dashboard — numbers first, always

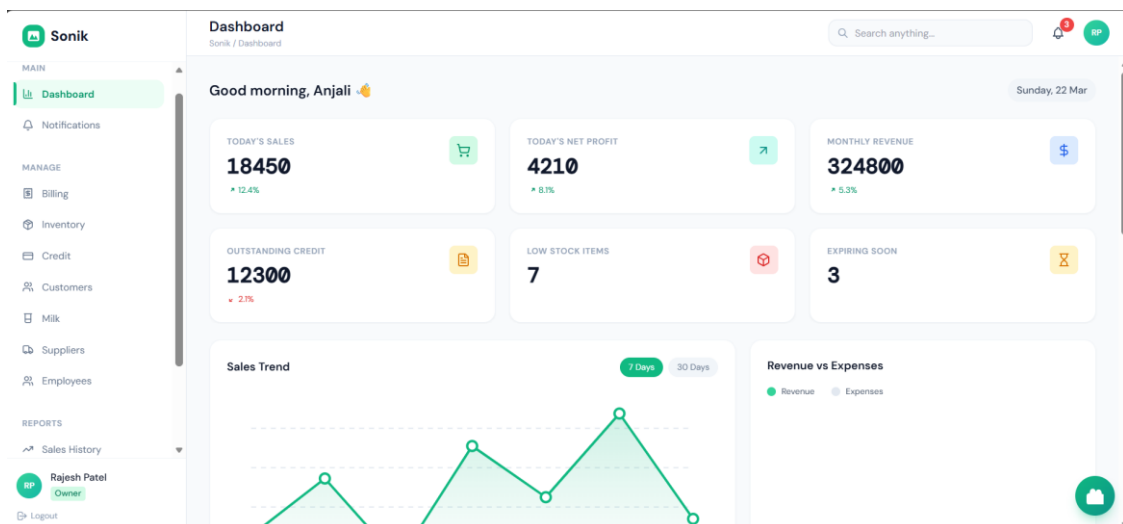


Fig 6.2 — Owner's Dashboard: KPI tiles, sales trend, and revenue vs. expenses.

KPI tiles (today's sales, net profit, monthly revenue, outstanding credit, low stock, expiring soon) sit above the fold, directly answering the “no profit visibility” and “no inventory visibility” findings from user research — the owner sees the notebook’s replacement the moment the dashboard opens.

Billing (POS) — shortened after real usage

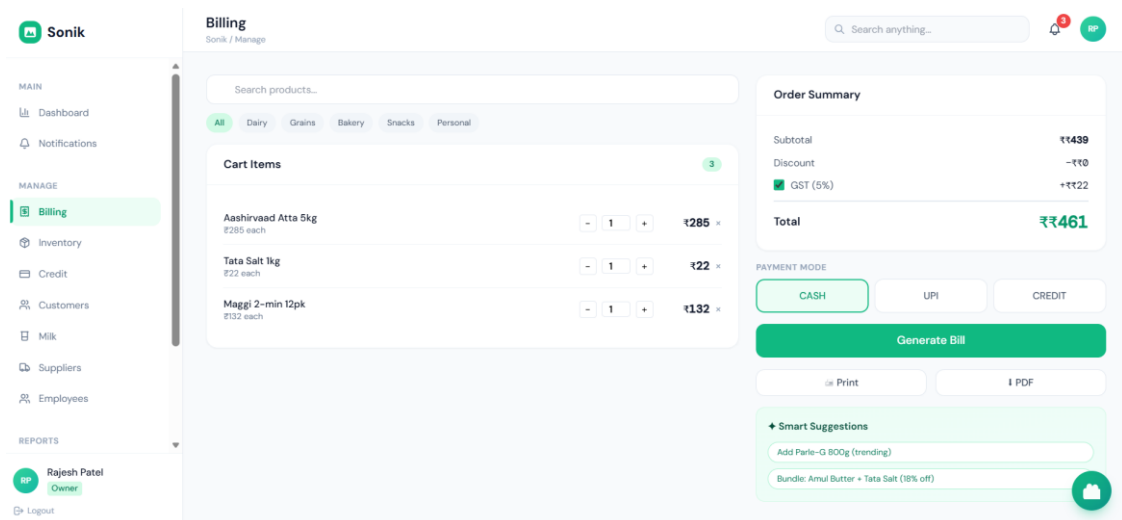


Fig 6.3 — Billing page with cart, GST calculation, and payment mode.

Originally, generating a bill redirected the user to Sales History to find a receipt — the store owner called this “too slow during busy hours.” A success modal with instant Print / PDF / Details actions was added directly on this page in the next milestone in response.

Inventory — expiry and reorder fields added directly against research

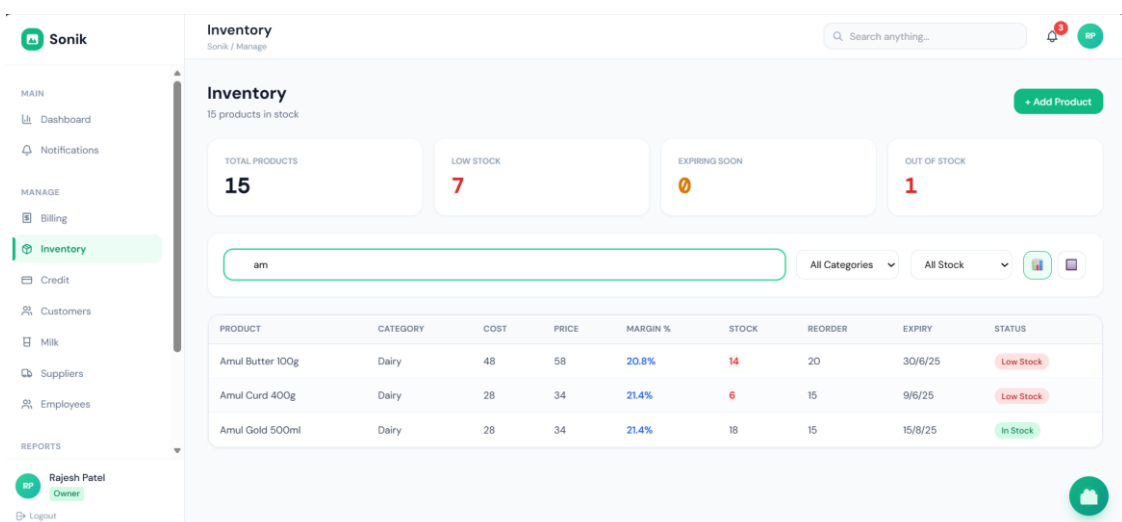


Fig 6.4 — Inventory list with low-stock, expiring, and out-of-stock counts.

The interview finding “no expiry tracking, no reorder mechanism” translated directly into the Reorder Level and Expiry Date fields on the Add Product form — not a generic notes field.

Credit Management — risk made visible

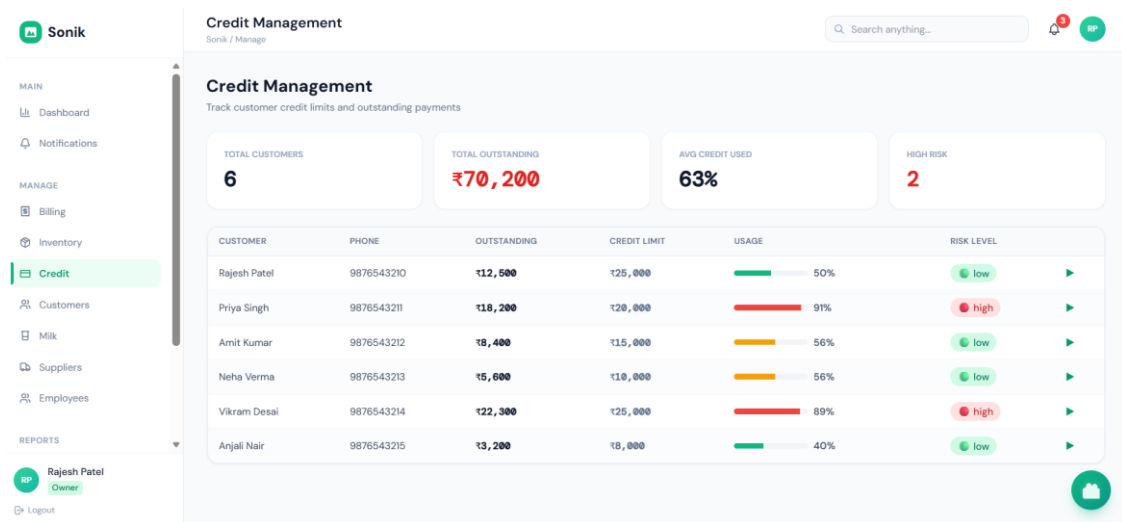


Fig 6.5 — Credit Management: outstanding balances, usage, and a Low/High risk tag per customer.

A simple Low/High risk tag was placed next to each customer's credit usage, so the “1–2% bad debt” and “delayed payments” pain points become something the owner can act on at a glance rather than something buried in a balance column.

Sales History — built for future accountability

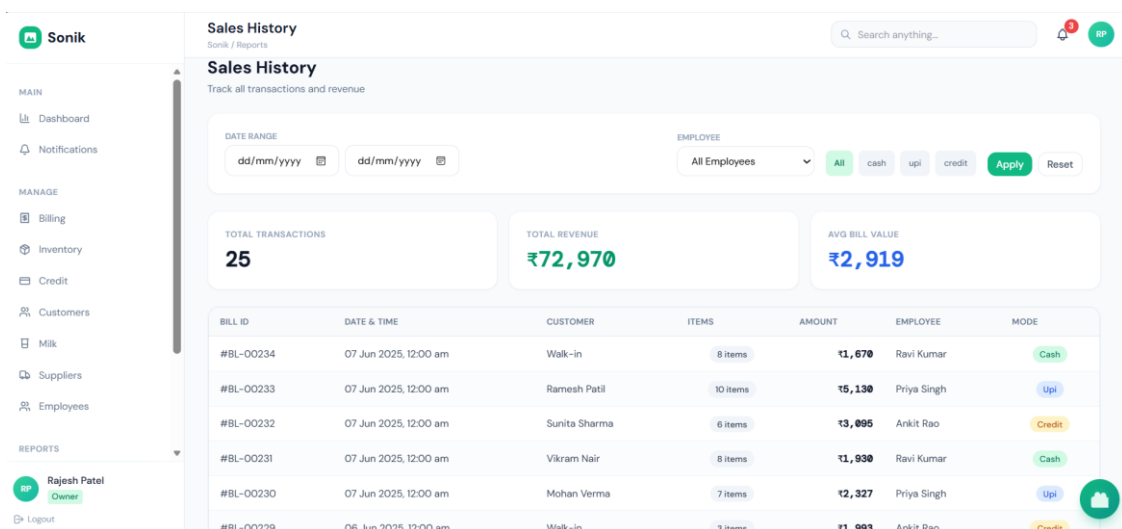


Fig 6.6 — Sales History, filterable by employee and payment mode.

Filtering by employee was included even before shift-based accountability was fully built, so that “sales grouped by employee shift” (a documented future user story) has a data trail to work from once it ships.

07. Feature Prioritisation

Prioritisation followed MoSCoW, sequenced against the two build sprints actually run (Milestone 3 and Milestone 4):

Priority	Features	Why
Must have	Auth, Billing/POS, Inventory (stock + expiry), basic customer & credit, Dashboard KPIs	These were the daily, hourly pain points surfaced directly in user research — built in Sprint 1
Should have	Live data everywhere (no mock data), Razorpay for supplier payments, instant PDF/print receipts, AI assistant upgraded to SQL-grounded answers, quick-suggestion chips, downloadable PDF reports	Real-user testing in Sprint 2 showed these removed the most daily friction once the core workflow existed
Could have	Employee shift/attendance tracking, sales-forecasting ML model, anomaly detection for theft/fraud, generative report narratives	Valuable but not blocking — documented as planned enhancements, not built by Milestone 5
Won't have (this phase)	Cloud deployment (Docker/AWS/GCP), multi-store support, JWT auth	Out of scope for a single-store, timeline-bound student build; explicitly flagged as next steps

Why this order, specifically: inventory, billing, and credit were the pain points Vinod described unprompted in the interview — they became Sprint 1. AI querying and payment automation looked like “nice to have until the notebook problem is solved,” but Milestone 4 user testing showed they were actually the biggest remaining daily-friction removers, so they were pulled forward into Sprint 2 instead of being pushed to a hypothetical V3.

08. PRD Highlights

The full PRD lives in the Milestone 1 and Milestone 5 reports. The excerpt below shows a few user stories traced all the way to the automated test that verifies them — the actual acceptance criteria the team shipped against.

User story	Acceptance criteria (from the test suite)
As an employee, I want to generate a bill so inventory updates automatically.	test_create_sale_authenticated → 200 OK, sale object returned with bill_id, total_amount, items, status: completed
As an owner, I want to log damaged or expired goods so losses are reflected.	test_damage_loss_record → 200 OK, loss entry recorded against the product
As an owner, I want to see customers overdue by 30+ days.	GET /api/transactions/report/credit-aging → 200 OK, aging buckets returned
System requirement: stock can never go negative.	test_stock_adjustment_negative_below_zero → 400 Bad Request, “Stock cannot go below zero”
System requirement: every write requires an authenticated session.	test_negative_missing_header → 401 Unauthorized when X-User-ID is absent

Selected non-functional requirements

- Soft-delete, not hard-delete, on inventory — discontinued products keep historical sales intact.
- Request/response validation via Pydantic v2 on every endpoint.
- SQLite for development and test isolation; MySQL-compatible for production via a single SQLAlchemy abstraction layer.
- Every API operation mapped to a user-story category in the OpenAPI spec for traceability (60 paths / 78 operations).

09. AI Feature

Why AI

Small retailers don't have access to enterprise analytics, so decisions get made on intuition. The goal was to bring AI-driven insight to a single grocery shop, not just digitise its notebook.

What problem required AI

The owner and manager needed plain-English answers grounded in their own data — “how many units of rice did I sell last week?” — not generic retail advice.

What it does

The AI Assistant takes a natural-language question and returns both the generated SQL and the exact result rows from the live database. The dashboard also surfaces proactive AI Insight cards (Demand Rising, Stock Critical, Profit Boost, Expiry Alert) and quick-suggestion chips (Today's Summary, Low Stock Items, Profit Analysis, Credit Risk Customers) so a first-time user knows what to ask.

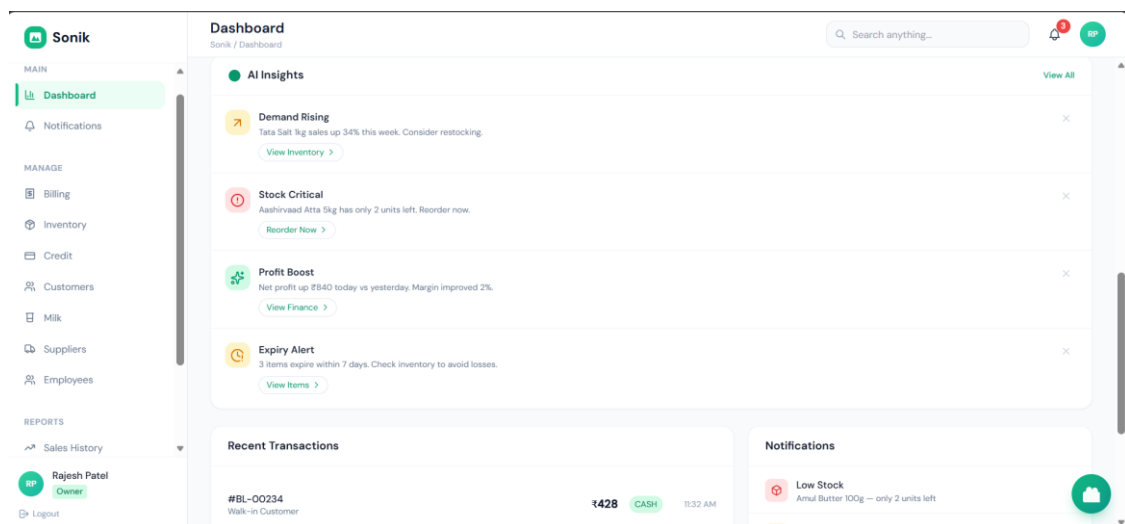


Fig 9.1 — AI Insight cards and recent transactions on the Owner's Dashboard.

AI risks considered

Risk	How it showed up / was handled
Hallucination / inexact answers	The original general-purpose chatbot gave estimates instead of exact figures — this is what forced the move to a SQL-grounded pipeline (see Section 11).
Unsafe / destructive outputs	The SQL layer enforces a hard SELECT-only guard, rejecting INSERT / UPDATE / DROP even if the model tries to generate one.
Latency & cost	Each question is a live LLM call plus a DB round-trip — not yet load-tested under concurrent use.
Data privacy	The assistant reads the same live store database as the rest of the app, so it's scoped by the same login/session as everything else — no separate, wider-access path.

10. AI Architecture

The AI Assistant Module was planned early and rebuilt once, after evaluation showed the first version wasn't good enough (Section 11). Only the components actually shipped are shown below — there is no separate vector database; the “knowledge base” is the live relational schema itself, injected directly into the LLM prompt.

Planned (Milestone 2)

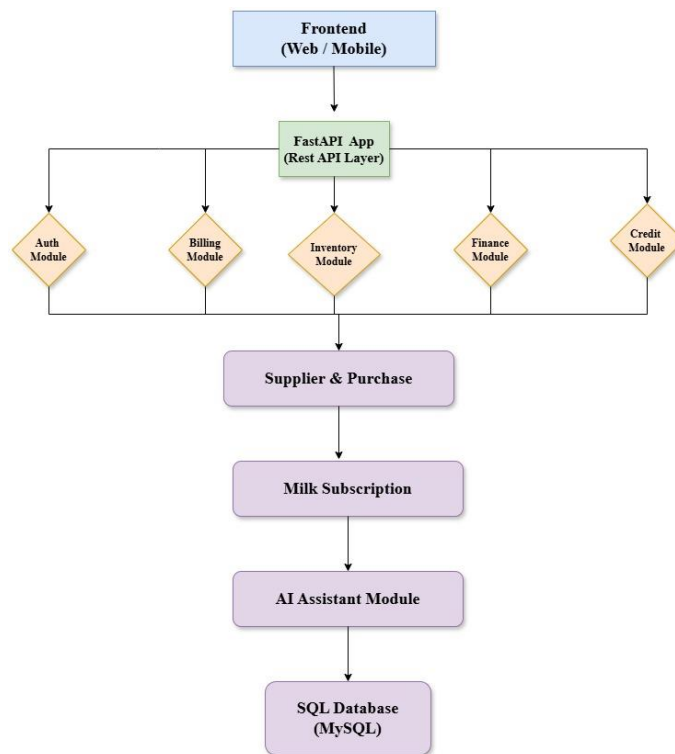


Fig 10.1 — Original component plan: the AI Assistant Module sits between the operational modules and the database.

As shipped (Milestone 5)

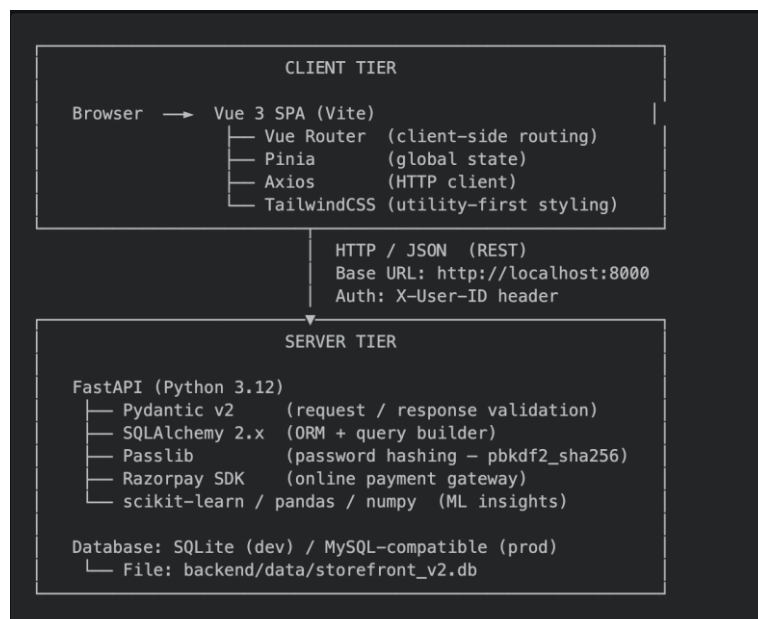


Fig 10.2 — Client/server architecture actually running: Vue 3 SPA ↔ FastAPI ↔ SQLite/MySQL.

The SQL-generation pipeline

```
User types a question in plain English
  | (AIChatModal.vue / dashboard "ask more")
  v
POST /query/sql ---> Retail-Assistant-Chatbot service
  |
  v
Groq LLM, prompted with the full database schema,
returns ONLY a SQL statement
  |
  v
handle_sql_query(): strips markdown fences,
enforces a SELECT-only safety guard
  |
  v
Query executes against the shared SQLite / MySQL database
  |
  v
Generated SQL + result rows returned to the
chat modal and dashboard (shown side by side)
```

11. Evaluation & Metrics

Evaluating the AI feature

Question	Answer
What was tested?	Whether the assistant's answers matched what a hand-written SQL query would return, not just whether the reply sounded plausible.
What counted as a good answer?	An answer backed by exact rows from the live database — same numbers a manual query would give.
What failure was found?	The first, general-purpose chatbot gave approximate estimates instead of exact figures for data questions (e.g. units sold last week).
What changed?	The querying approach was rebuilt around a constrained natural-language-to-SQL pipeline with a SELECT-only guard, specifically to close that accuracy gap.

Evaluating the system (API test suite)

	Sprint 1 (Milestone 3)	Sprint 2 (Milestone 4 / 5)
Endpoints tested	25, across 7 functional domains	60 paths / 78 operations
Result	19 passed / 6 failed	84 / 84 test cases passed (100%)
Failure pattern	All 6 failures traced to 3 root causes: ORM/DB schema mismatch, a Pydantic response-key mismatch, and an INSERT column mismatch	Root causes fixed; regression suite added, incl. a soft-delete guard test

Metrics

Verified: 84/84 (100%) automated test pass rate at final submission, across positive, negative, and integration flows (full sale flow, customer credit flow); 60 paths / 78 operations fully documented in OpenAPI with per-endpoint user-story mapping.

Target / proposed impact (not yet production-validated): the team's own presentation frames these as goals to validate once a real store uses the system, not confirmed outcomes.

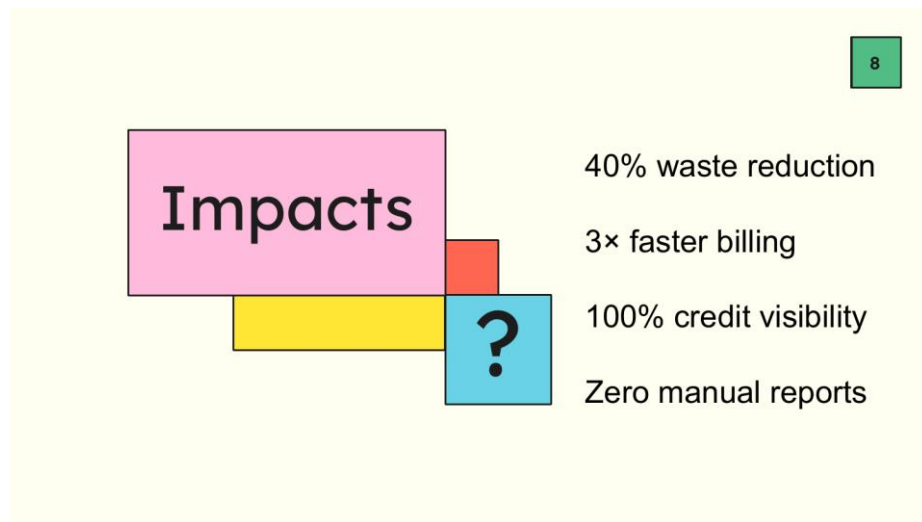


Fig 11.1 — Proposed impact goals from the project pitch — marked with a “?” in the original deck, i.e. targets to validate, not measured results.

12. Challenges & Trade-offs

Challenge 1 — ORM/DB schema drift broke 6 endpoints

Decision: treat the live database as the source of truth and align the ORM to it, rather than the reverse.

Trade-off: extra rework in Sprint 2 that schema-first design from day one would have avoided — a lesson carried forward.

Challenge 2 — A milestone deadline vs. clean backend structure

Decision: keep core routes in one ~2000-line main.py instead of splitting into router modules (the ML sub-package is the one exception). **Trade-off:** faster to build and everything stays discoverable in one place now, at a real cost to long-term maintainability.

Challenge 3 — Deleting a product must not break historical sales

Decision: soft-delete products (status = “discontinued”) instead of removing the row. **Trade-off:** discontinued products still appear in raw inventory queries unless explicitly filtered — the test suite had to specifically guard this behaviour.

Challenge 4 — Auth needed to exist quickly

Decision: use a simple X-User-ID integer header instead of JWT. **Trade-off:** much less implementation overhead for a class-project timeline, but the team explicitly flagged a JWT migration as required before this could be a real production system.

Challenge 5 — Local dev speed vs. production-grade database

Decision: build on SQLite behind a SQLAlchemy abstraction so only the connection string changes for MySQL. **Trade-off:** any MySQL-specific behaviour differences would only surface after the switch — a risk accepted for development speed.

13. Results & Learnings

Results

Sonik was designed, built, and tested across five milestones between February and April 2026, ending with a fully integrated Vue 3 + FastAPI system: live Razorpay payments for suppliers and customers, instant PDF/print receipts, downloadable PDF business reports, and an AI assistant that answers natural-language questions against the live database — backed by a 100% (84/84) automated test pass rate.

Learnings

- Schema-first discipline matters: every one of Sprint 1's 6 failing endpoints traced back to ORM/DB drift, not business logic — worth catching before writing routes, not after.
- Real user sessions surfaced fixes internal review didn't: all five Milestone 4 changes (Razorpay for suppliers, instant receipts, SQL-grounded AI, suggestion chips, PDF reports) came from watching an actual store owner, cashier, and manager use the prototype — not from the team's own backlog.
- “AI feature” isn't one feature: users didn't want a chatbot personality, they wanted trustworthy numbers — the highest-leverage AI change was constraining the model to generate SQL against real data, not improving its prose.

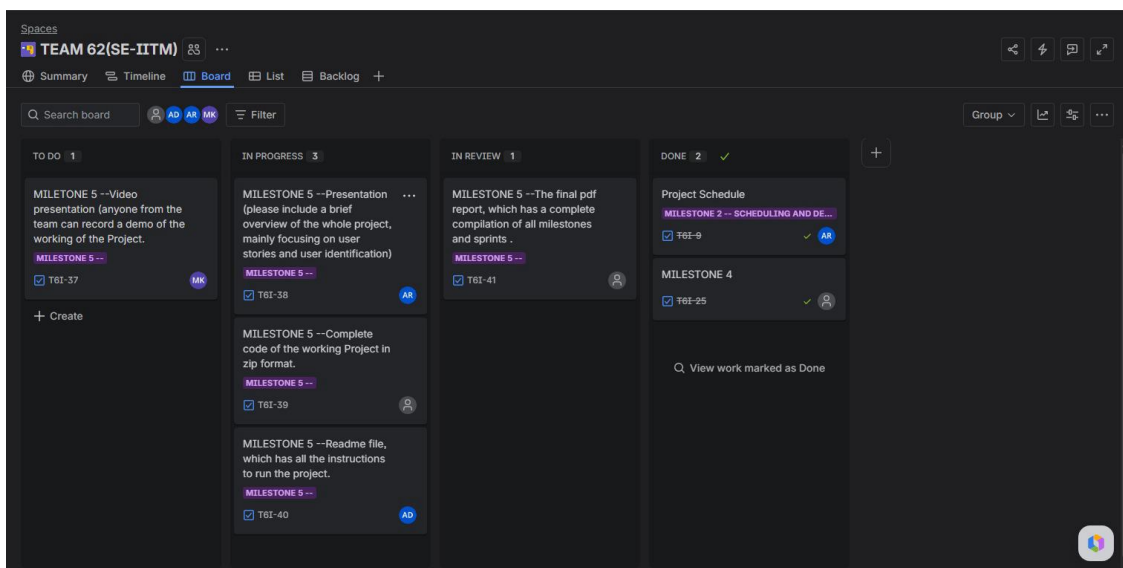


Fig 13.1 — Milestone 5 board at submission — presentation, packaging, and final report moving through review to done.

14. Future Improvements (V1 → V2)

Carried over from the original problem statement and the Milestone 1 roadmap — identified early, not yet built:

- Employee shift/attendance system beyond basic login — linking billing errors and performance to specific staff.
- A trained sales-forecasting model (Linear Regression / Random Forest / LSTM) for demand prediction — planned in the original Problem Statement, not yet a production model.
- Anomaly detection for unusual sales or inventory fluctuations (theft, spikes, recording errors) — a documented user story, not yet implemented.
- Automated, generative business-report narratives, beyond the current client-side PDF export of raw metrics.
- Cloud deployment (Docker / AWS / Azure / GCP) and JWT-based auth — both flagged internally as required before this could run as a real multi-store product.
- Formal AI accuracy testing across a larger question set, plus latency/cost benchmarking under concurrent use.

15. My Contribution

My role on the team centred on the AI/data feature line and cross-team documentation, alongside general PM-lifecycle work — research synthesis, prioritisation, and sprint tracking — for the wider team. Per the team's own sprint records and Kanban boards:

- **AI Feature Planning** (Sprint 1, Milestone 2, with Navirah Mohsin) — scoped which AI capabilities (chatbot + ML insights) would go into the system, and when.
- **Chatbot Development — Backend & Logic** (Sprint 2, Milestone 2, sole ownership) — designed the conversation flow and knowledge-base integration that became the AI Assistant Module.
- **Chatbot Frontend Development** (Sprint 2, Milestone 2, with Navirah Mohsin) — built the chat interface that evolved into AIChatModal.vue.
- **Machine Learning Model Development Planning** (Sprint 2, Milestone 2, with Navirah Mohsin).
- **Documentation** (Sprint 2, Milestone 2, with Navirah Mohsin) — system architecture and implementation write-ups.
- **Milestone 5 final presentation** — owned end-to-end per the team board (“include a brief overview of the whole project, mainly focusing on user stories and user identification”).
- **Version control** — authored and merged pull requests into the shared repository over the project timeline.

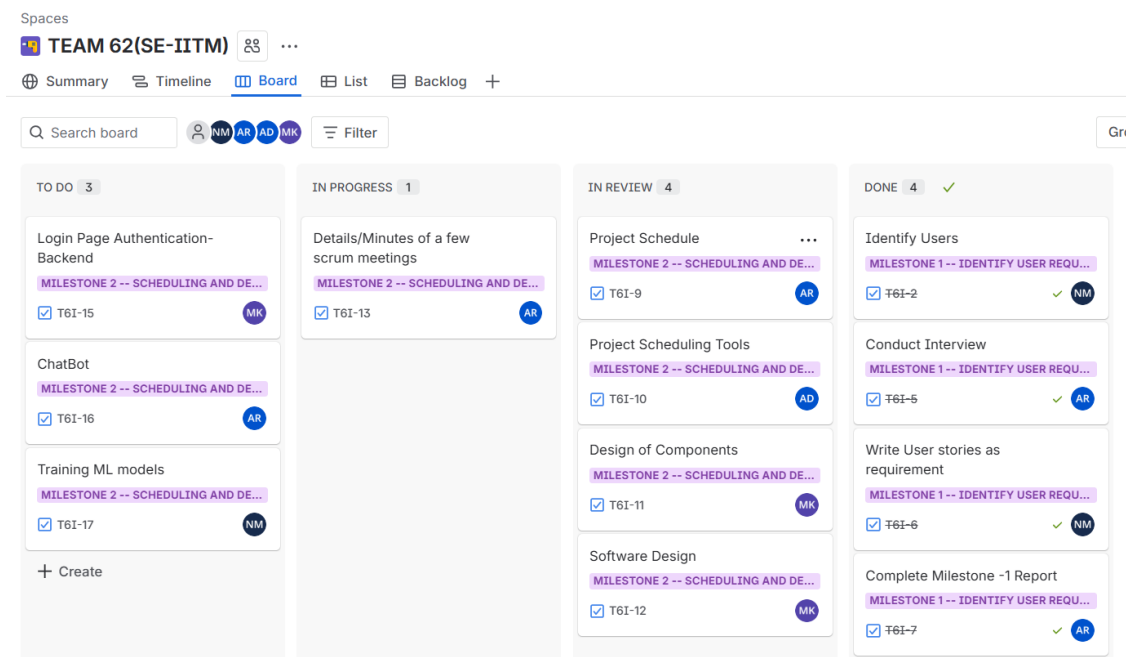


Fig 15.1 — Milestone 2 board: ChatBot and AI Feature Planning tasks assigned to AR (Aanchal Routela).